

达尔文哥德尔机器：自我改进智能体的开放式演化

Jenny Zhang Shengran Hu Cong Lu Robert Lange Jeff Clune

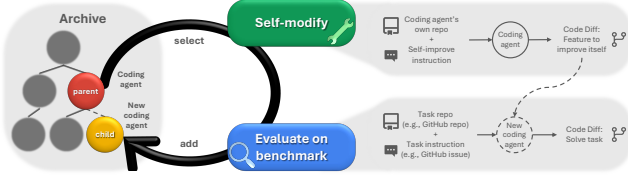


图 1: DGM 在自我修改与下游评测之间交替，并以开放式探索从档案中选择智能体。

摘要

当代 AI 系统的架构由人类固定设计，无法自主、持续地改进。哥德尔机器提出了理论上的替代方案：反复以可证明有益的方式修改自身；但在实践中几乎不可能证明绝大多数修改的净收益。本文提出**达尔文哥德尔机器** (DGM)，它迭代修改自身代码，并以编程基准的经验结果验证改动。DGM 受达尔文演化与开放式研究启发，维护已生成编程智能体的档案库：从中采样一个智能体，让基础模型创建其新颖版本，由此形成多样、高质量智能体不断增长的树，并行探索搜索空间中的多条路径。实验中，DGM 自动发现更好的代码编辑工具、长上下文管理和同行评审机制，使 SWE-bench 成绩从 20.0% 提高到 50.0%，Polyglot 从 14.2% 提高到 30.7%，显著优于没有自我改进或开放式探索的基线。实验采用沙箱和人工监督等安全措施。

1 引言

科学进步是累积且开放的：每项创新都成为未来发现的垫脚石。相比之下，当前 AI 通常只能在预先设定的边界内学习，不能改写自己的源代码，进步节奏仍高度依赖人工干预。本文研究能否安全地自动搜索更好的 AI，使系统既能解决下游问题，也能提升自己修改自身的能力。

哥德尔机器要求形式证明每次改写都有益，实际不可行，因为工具、提示和任务上下文的影响取决于具体模型。DGM 放弃先验形式证明，转而在编程基准上经验验证候选改动。这类似生物演化：突变先产生、试验，再由选择保留。DGM 不是沿一条单一路径爬山，而是保留已发现的智能体档案，让暂时次优但有趣的变体成为日后突破的垫脚石。

2 相关工作

开放式系统追求持续产生新颖且可学习的产物，已有质量-多样性、目标导向探索和内在动机等路径；近期基础模型可作为生成和评估新行为的通用引擎。基于基础模型的元学习还可优化提示、工作流、工具和智能

体模块，但通常依赖固定、人工设计的元智能体。已有自我改进研究覆盖权重参数化、元训练优化器、自博弈及递归修改的基础模型智能体。DGM 的区别在于：同一个系统既解决编程任务又改进自己的代码库，并用开放档案避免只追逐即时收益而停在局部最优。

3 达尔文哥德尔机器

DGM 将“可证明改进”放宽为“由实验支持的改进”。系统从一个编程智能体开始；该智能体是由冻结的预训练基础模型驱动、能读写和执行代码的代码仓库。因为 Python 等通用图灵完备语言可表达任意可计算过程，修改其代码库即可改变提示、工具与工作流。本文只研究冻结基础模型上的编程智能体设计，而非训练新的基础模型。

每轮分为自我修改和评测。档案中的父智能体分析自己的评测日志、提出下一项待实现特性，再编辑自身代码库产生子智能体；子智能体在编程基准上定量评测。只有能成功构建且仍能编辑代码库的版本被加入档案，其他版本丢弃。父节点采样大致正比于性能、反比于已有具备编辑能力的子节点数，因此兼顾强者利用和欠探索谱系的探索，且所有智能体保留非零被选概率。当前档案维护和父节点选择本身仍是固定的，未交给 DGM 修改。

4 实验设置

初始智能体仅配备 Bash 执行工具和目录/文件查看编辑工具，以展示最小起点上的提升。DGM 运行 80 次迭代；SWE-bench 中每两次并行，Polyglot 中每四次并行。自我修改使用 Claude 3.5 Sonnet；评测阶段 SWE-bench 使用同一模型，Polyglot 使用 o3-mini。

评测覆盖 SWE-bench Verified 与多语言 Polyglot。前者常需改动多个 Python 文件；后者多为单文件实现但仍要求理解仓库。为控制成本，系统分阶段评测：先用 10 个任务检查基本代码编辑能力，通过后扩展到 50 个任务；SWE-bench 的强候选进一步测 200 个任务。比较对象包括不自我改进的 DGM、无开放式探索的 DGM（总从最新可用版本继续改），以及相同子集上的开源方案。

5 结果与分析

DGM 在 SWE-bench 从 20.0% 提升到 50.0%，在 Polyglot 从 14.2% 提升到 30.7%。相较固定基智能体不断生成新版本的基线，持续自我改进带来更持久的进展；相较只沿最新版本演化的基线，保留全部档案能发现更优智能体，说明早期变体可在很久以后发挥作用。

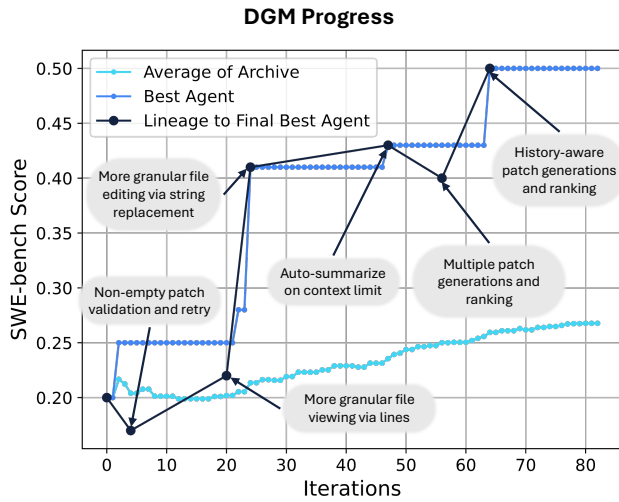


图 2: DGM 的能力随迭代进展；档案中的分支提供多条可继续演化的路径。

发现的能力包括更可靠的编辑工具、长上下文窗口管理和同行评审。这些特性还能在不同基础模型、基准和任务之间迁移。消融说明自我改进与开放式探索缺一不可：前者让提升下游编程能力直接反映更强的未来自我修改能力，后者则避免单一谱系因偶然退化或局部最优而阻断搜索。

6 安全与局限

自我修改系统存在滥用、失控执行和评测过拟合风险。作者将实验置于沙箱内，保留每次改动的可追溯记录，并实行人工监督。经验基准改进不是对所有场景收益的证明，且编程基准分数只是自我改进能力的代理指标。本文也未让系统训练或修改基础模型，更没有让开放式探索机制本身自我修改；这些都是未来工作，同时要求更严格的安全保障。

7 结论

DGM 以经验验证替代不可行的形式证明，通过档案驱动的开放式探索，让编程智能体递归改进自身代码。结果显示，持续自我改进和多样化垫脚石可共同带来显著、可迁移的编程能力提升，为安全研究自我改进系统提供了一个具体起点。